Updates on the FROST NIST Submission

Chelsea Komlo

University of Waterloo,
NEAR One

January 26, 2026

FROST NIST Submission Team



Elizabeth Crites

Conrado Gouvea

Ian Goldberg

Jack Grigg

Jonathan Katz

Chelsea Komlo

Mary Maller

Simon Rastikian

Stefano Tessaro

Nikita Sorokovikov

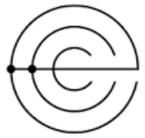
Denis Varlakov

Chenzhi Zhu









ELECTRIC COIN CO.









NISTIR 8214C (Draft)

NIST First Call for Multi-Party Threshold Schemes

Date Published: January 25, 2023

Comments Due: April 10, 2023

Email Comments to: nistir-8214C-comments@nist.gov

Author(s)

Luís T. A. N. Brandão (Strativia), Rene Peralta (NIST)

Background

(Single-Party) Schnorr Signatures





To generate a key pair:

$$sk \xleftarrow{\$} \mathbb{Z}_q \; ; \; PK \leftarrow g^{sk}$$

(Single-Party) Schnorr Signatures





To generate a key pair:

$$sk \xleftarrow{\$} \mathbb{Z}_q \; ; \; PK \leftarrow g^{sk}$$

To sign a message m:

$$r \xleftarrow{\$} \mathbb{Z}_q \; ; \; R \leftarrow g^r$$

$$c \leftarrow H(PK, m, R)$$

$$z \leftarrow r + csk$$

(Single-Party) Schnorr Signatures



$$\sigma = (R, z)$$



To generate a key pair:

$$sk \xleftarrow{\$} \mathbb{Z}_q \; ; \; PK \leftarrow g^{sk}$$

To sign a message m:

$$r \xleftarrow{\$} \mathbb{Z}_q \; ; \; R \leftarrow g^r$$

$$c \leftarrow H(PK, m, R)$$

$$z \leftarrow r + csk$$

(Single-Party) Schnorr Signatures



$\sigma = (R, z)$



To generate a key pair:

$sk \xleftarrow{\$} \mathbb{Z}_q \; ; \; PK \leftarrow g^{sk}$

To sign a message m:

$r \xleftarrow{\$} \mathbb{Z}_q \; ; \; R \leftarrow g^r$

$c \leftarrow H(PK, m, R)$

$z \leftarrow r + csk$

To verify (PK, σ, m):

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c \stackrel{?}{=} g^z$

output accept/reject

Multi-Party Schnorr Signatures

How to share r ?

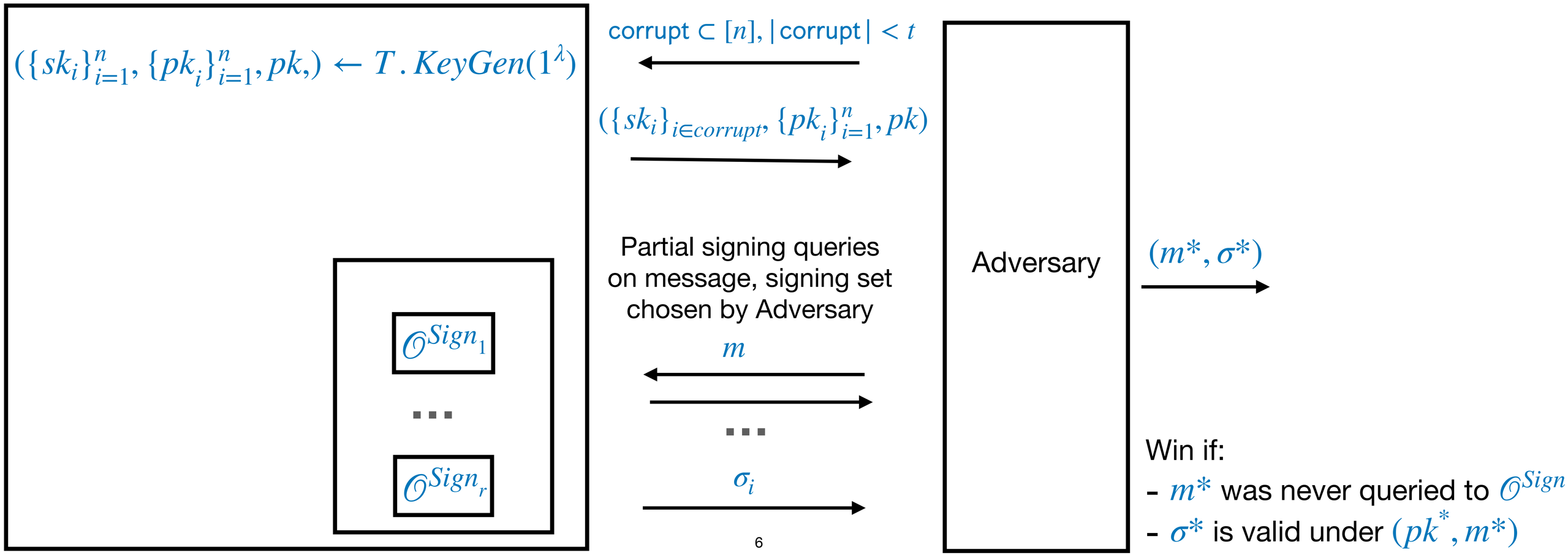
How to share sk ?

$$z \leftarrow r + c \cdot sk$$

$$sig = (R, z)$$

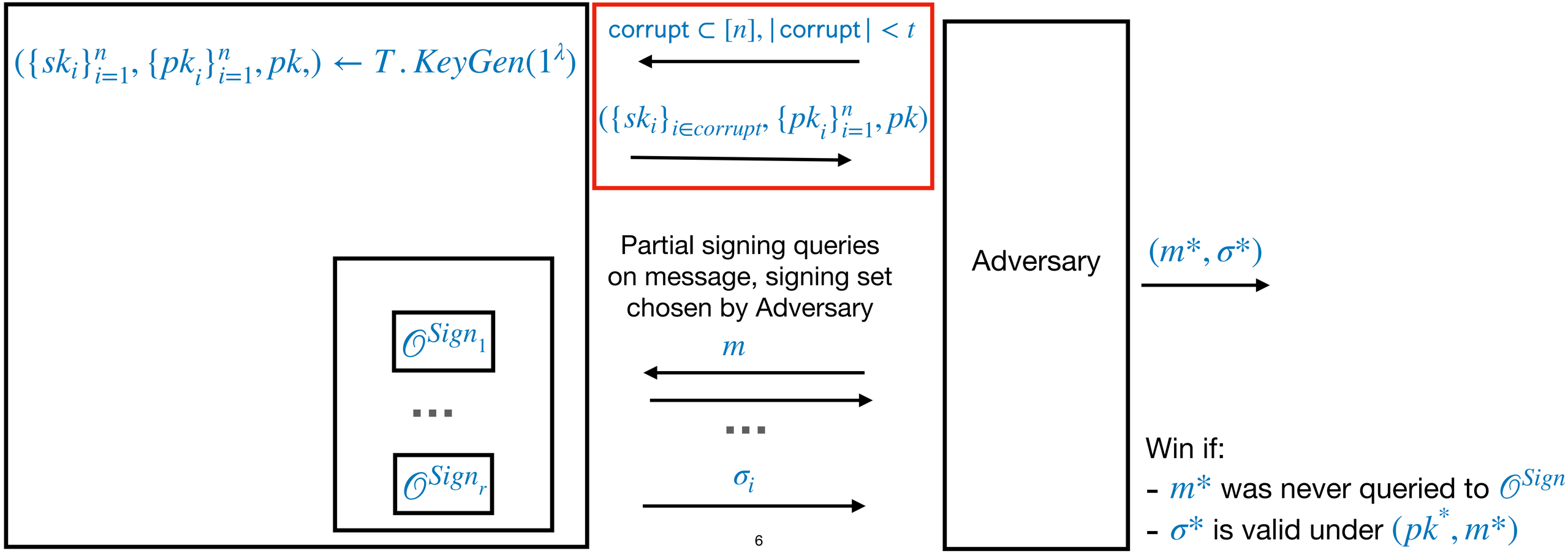
Static Unforgeability

A threshold signature scheme T is secure if no PPT adversary can win the following game with non-negligible advantage:



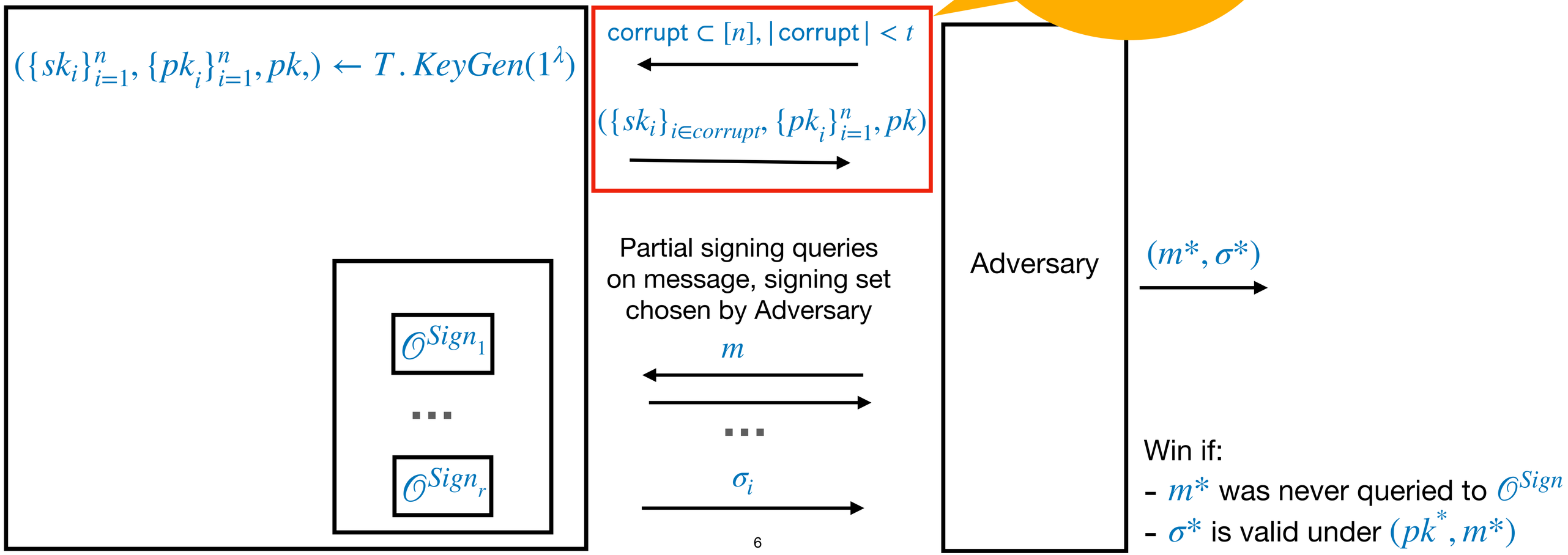
Static Unforgeability

A threshold signature scheme T is secure if no PPT adversary can win the following game with non-negligible advantage:



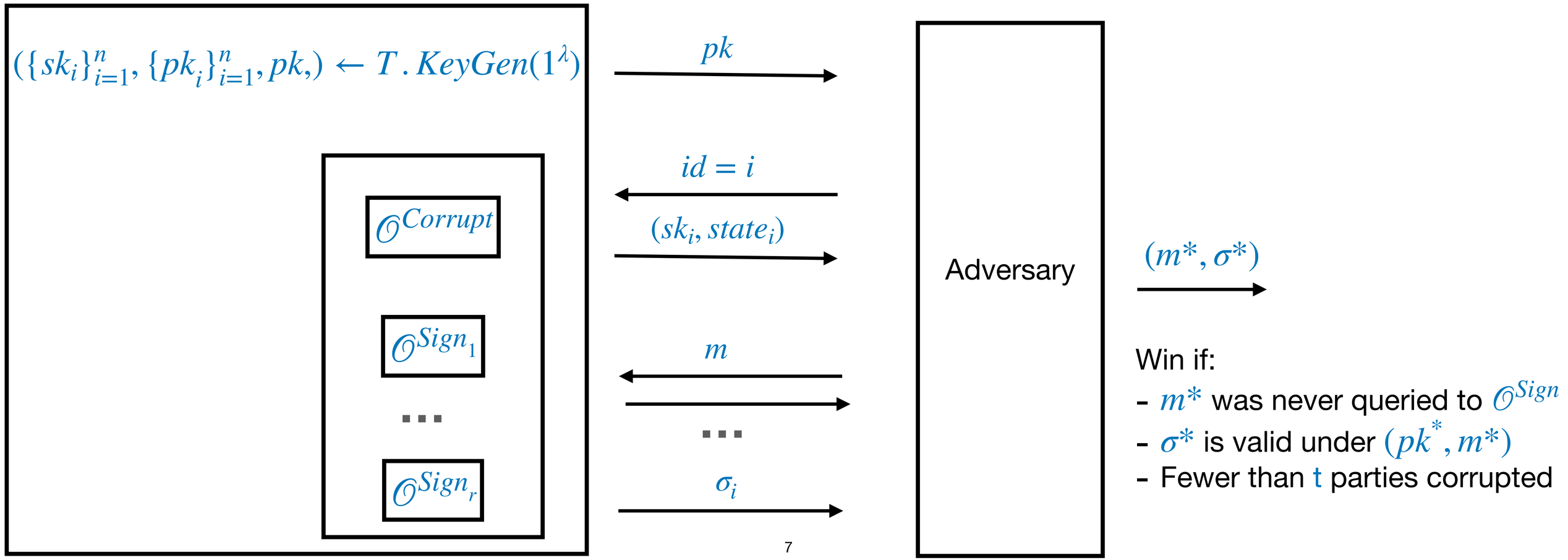
Static Unforgeability

A threshold signature scheme T is secure if no PPT adversary can win the following game with non-negligible advantage:



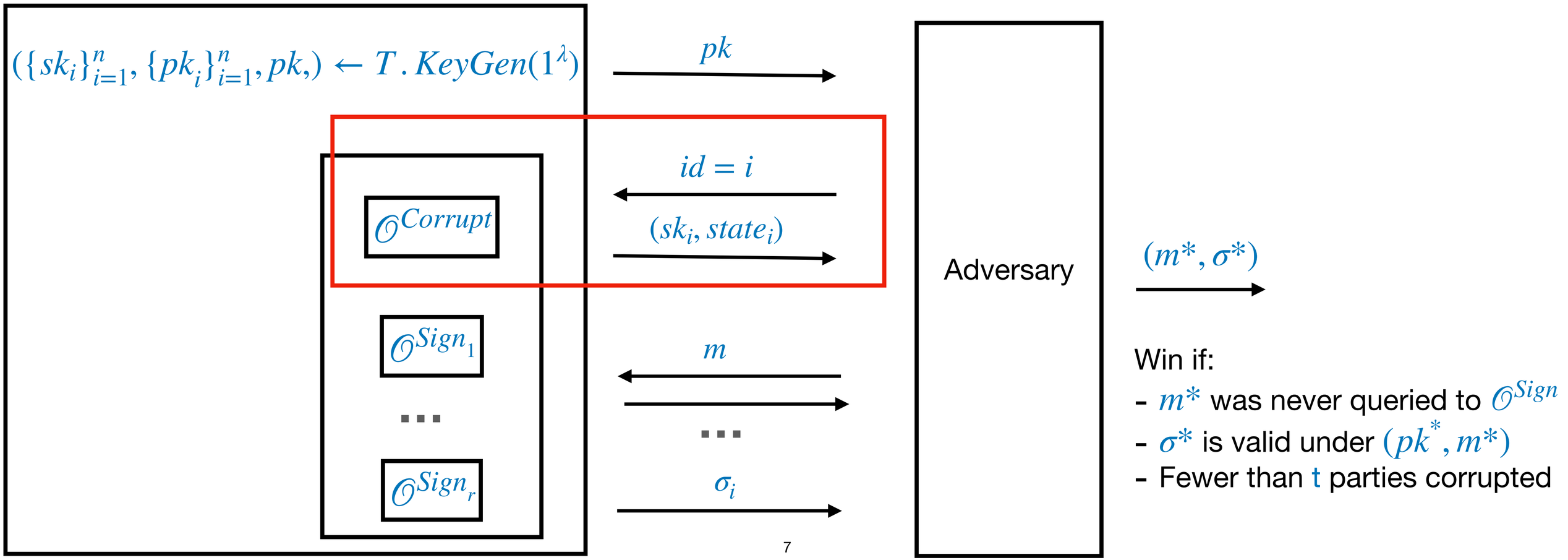
Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



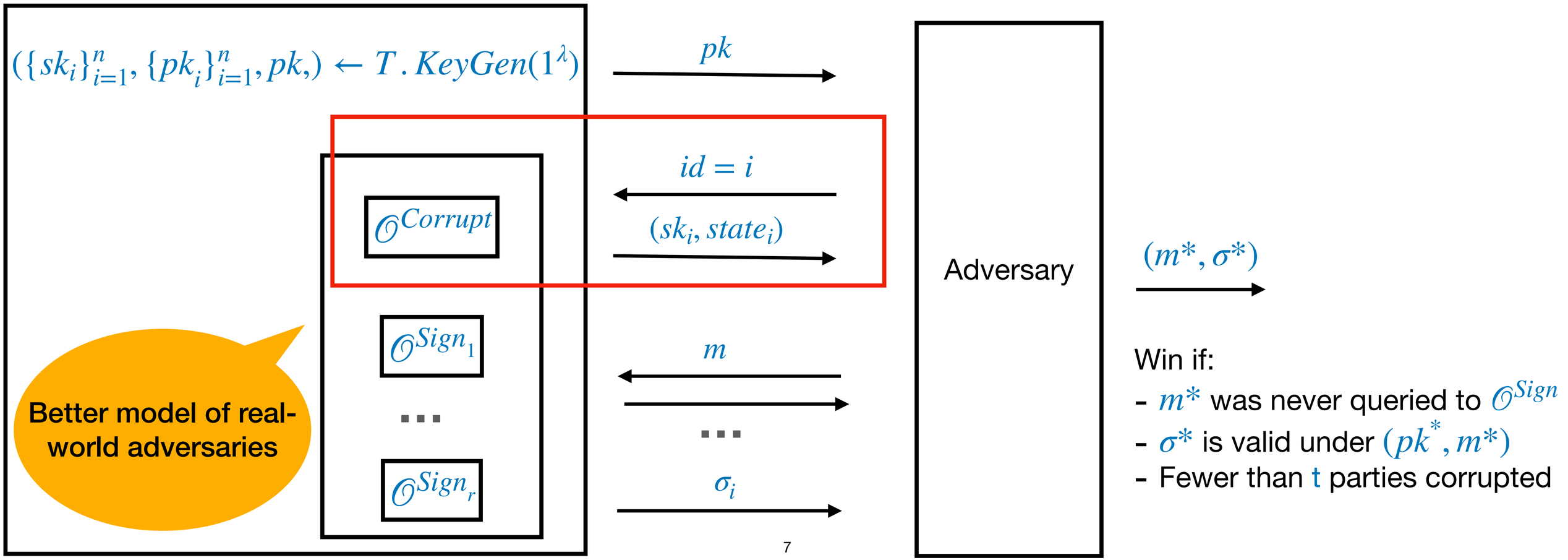
Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



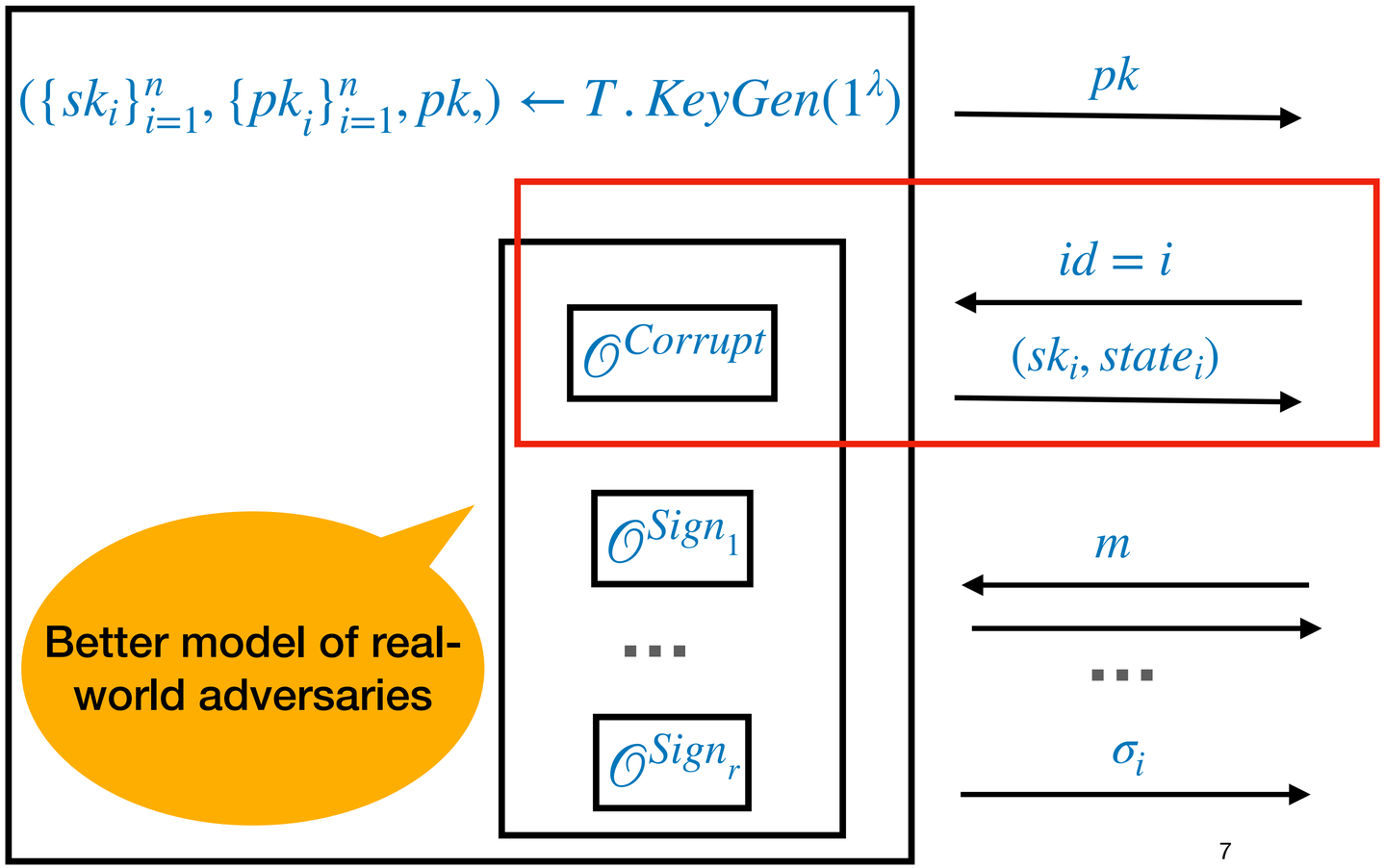
Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



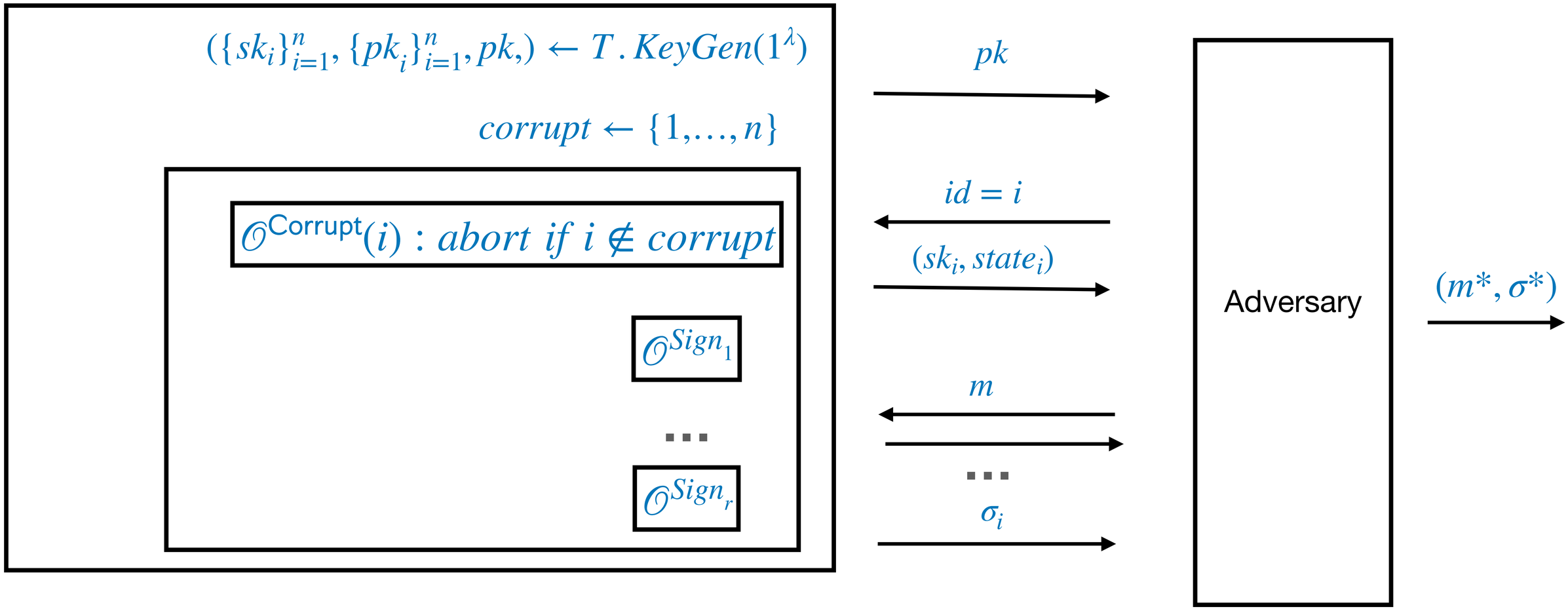


Environment must simulate:
1) Secret key shares,
2) Signing state
(over all open sessions)

(m^*, σ^*)

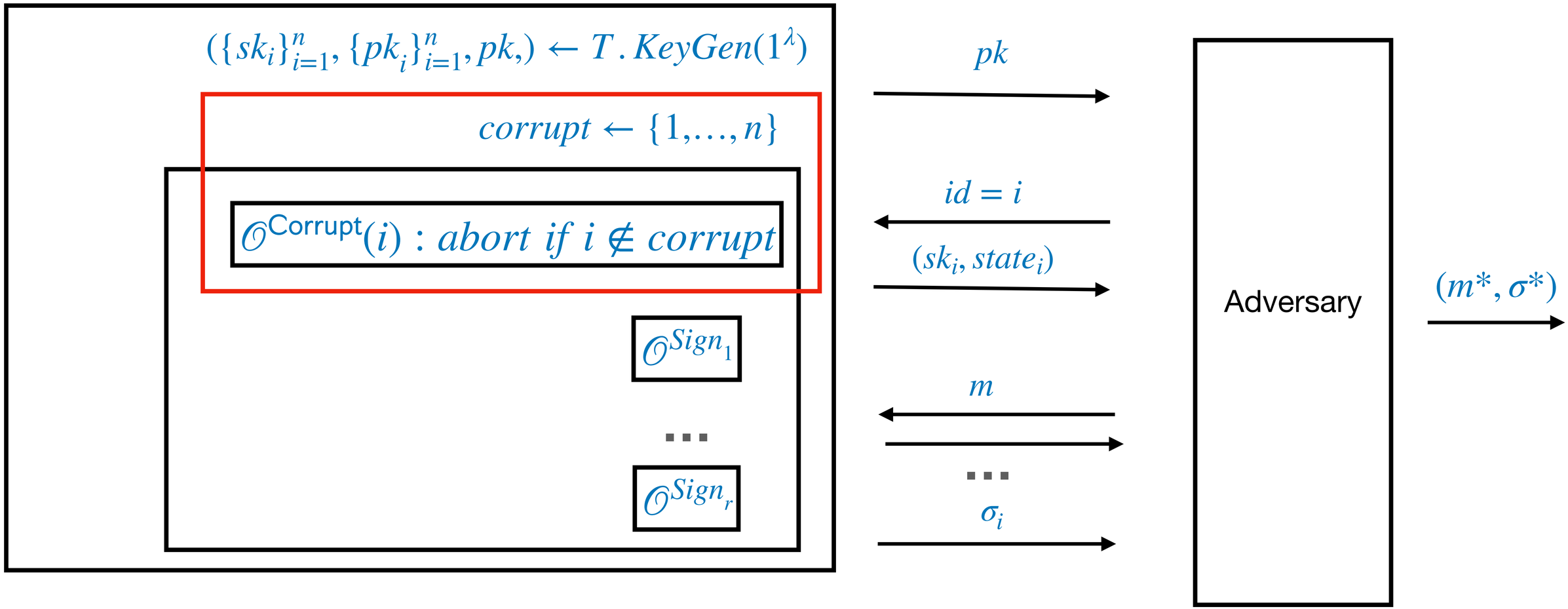
Win if:
- m^* was never queried to $\mathcal{O}^{Sign}$
- σ^* is valid under (pk^*, m^*)
- Fewer than t parties corrupted

Trivial Adaptive Unforgeability from Guessing



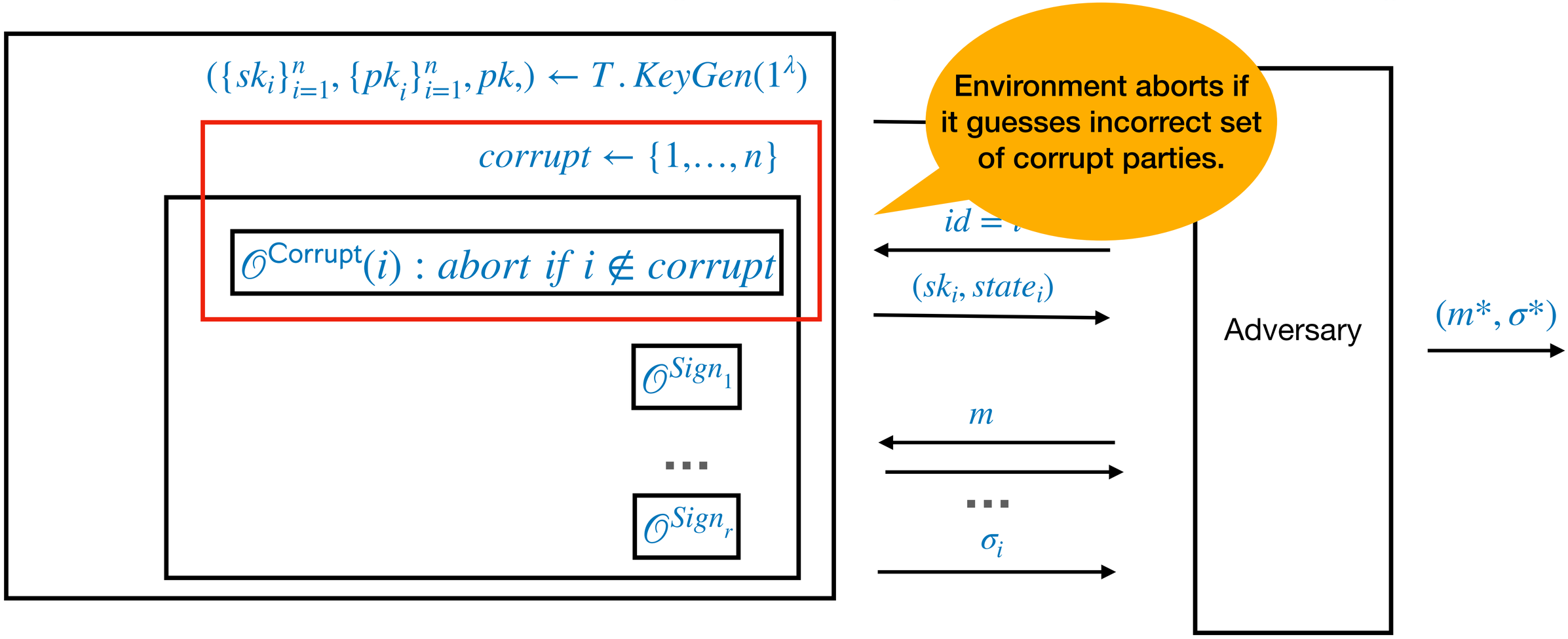
- Tightness loss of $\binom{n}{t}$ in number of parties and corruption threshold.

Trivial Adaptive Unforgeability from Guessing



- Tightness loss of $\binom{n}{t}$ in number of parties and corruption threshold.

Trivial Adaptive Unforgeability from Guessing



- Tightness loss of $\binom{n}{t}$ in number of parties and corruption threshold.

Is (Non-Trivial) Adaptive Security Meaningful in Practice?

Is (Non-Trivial) Adaptive Security Meaningful in Practice?

- Non-Trivial = Reduction succeeds with probability better than $\frac{1}{\binom{n}{t}}$

Is (Non-Trivial) Adaptive Security Meaningful in Practice?

- Non-Trivial = Reduction succeeds with probability better than $\frac{1}{\binom{n}{t}}$
- Currently no efficient attacks exist in the adaptive setting.

Is (Non-Trivial) Adaptive Security Meaningful in Practice?

- Non-Trivial = Reduction succeeds with probability better than $\frac{1}{\binom{n}{t}}$
- Currently no efficient attacks exist in the adaptive setting.
- More cryptanalysis is needed (see next talk) [CS25, CKKTZ25].

Prior Art

A First Attempt

PK_1





PK_2

PK_n



Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

A First Attempt

$r_1 \leftarrow \mathbb{Z}_q$

PK_1





PK_2

PK_n



Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



A First Attempt

PK_1



$r_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}$



PK_2

$\vdots$



PK_n

Combine/Verify

$z = \Sigma_{i=1}^{n} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

A First Attempt

PK_1



$r_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}$

R_1



PK_2



PK_n

Combine/Verify

$z = \Sigma_{i=1}^{n} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

A First Attempt

PK_1



$r_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}$

R_1



PK_2



PK_n

Combine/Verify

$z = \Sigma_{i=1}^{n} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

A First Attempt

PK_1



$r_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}$

R_1

$\{R_2 \ldots, R_n\}$



PK_2



PK_n

Combine/Verify

$z = \Sigma_{i=1}^{n} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

A First Attempt

PK_1



$r_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}$

R_1

$\{R_2 \ldots, R_n\}$



PK_2



PK_n

Combine/Verify

$z = \Sigma_{i=1}^{n} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

A First Attempt

PK_1



$r_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}$

$R = \prod_{i=1}^{n} R_i$

R_1

$\{R_2 \ldots, R_n\}$



PK_2



PK_n

Combine/Verify

$z = \Sigma_{i=1}^{n} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

A First Attempt

PK_1



$$r_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}$$

$$R = \prod_{i=1}^{n} R_i$$

$$c = H(PK, R, m)$$

R_1 →

← $\{R_2 \ldots, R_n\}$



PK_2



PK_n

Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



A First Attempt

PK_1



$$r_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}$$

$$R = \prod_{i=1}^{n} R_i$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + csk_1\lambda_1$$

R_1 →

← $\{R_2 \ldots, R_n\}$



PK_2



PK_n

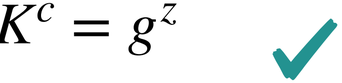
Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



A First Attempt

PK_1



$$r_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}$$

$$R = \prod_{i=1}^{n} R_i$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + csk_1\lambda_1$$

R_1 →

← $\{R_2 \ldots, R_n\}$

↓



PK_2



PK_n

Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



A First Attempt

PK_1



$$r_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}$$

$$R = \prod_{i=1}^{n} R_i$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + csk_1\lambda_1$$

R_1

$\{R_2 \ldots, R_n\}$

$(z_1, \ldots, z_n)$



PK_2

PK_n



Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



A First Attempt

PK_1



$$r_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}$$

$$R = \prod_{i=1}^{n} R_i$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + csk_1\lambda_1$$

R_1 →

← $\{R_2 \ldots, R_n\}$

↓

$(z_1, \ldots, z_n)$



PK_2



PK_n

Combine/Verify

$$z = \Sigma_{i=1}^{n} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

✓

Secure in a sequential setting

Concurrent Security: ROS

(Random inhomogeneities in a Overdetermined Solvable system of linear equations)

[DEFKLNS19, BLLOR21]

Session 1

Session k

sk_1



. . .



sk_2



Concurrent Security: ROS

Session 1

sk_1



$R_1^{(1)}$

...

Session k



$R_1^{(k)}$

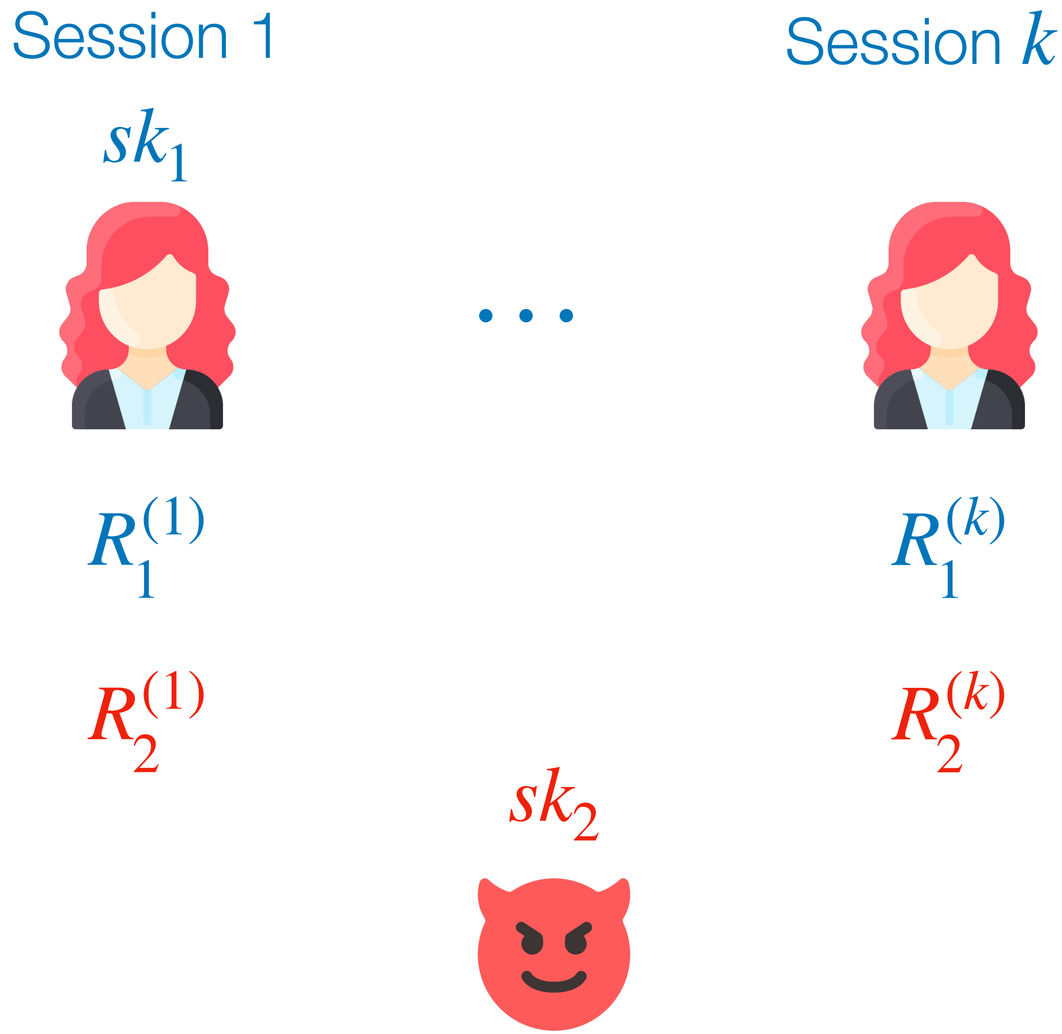
sk_2



(Random inhomogeneities in a Overdetermined Solvable system of linear equations)

[DEFKLNS19, BLLOR21]

Concurrent Security: ROS



(Random inhomogeneities in a Overdetermined Solvable system of linear equations)

[DEFKLNS19, BLLOR21]

Concurrent Security: ROS

(Random inhomogeneities in a Overdetermined Solvable system of linear equations)

[DEFKLNS19, BLLOR21]

Session 1

Session k

sk_1



$\cdots$



$R_1^{(1)}$

$R_1^{(k)}$

$R_2^{(1)}$

$R_2^{(k)}$

sk_2



Can forge!

Concurrent Security: ROS

Session 1

sk_1



$R_1^{(1)}$

$R_2^{(1)}$

. . .

Session k



$R_1^{(k)}$

$R_2^{(k)}$

sk_2



Can forge!

(Random inhomogeneities in a Overdetermined Solvable system of linear equations)

[DEFKLNS19, BLLOR21]

Affected:

- multi-signatures
- threshold signatures
- blind signatures

Three Round Fix: Force adversary to commit to its contributions.



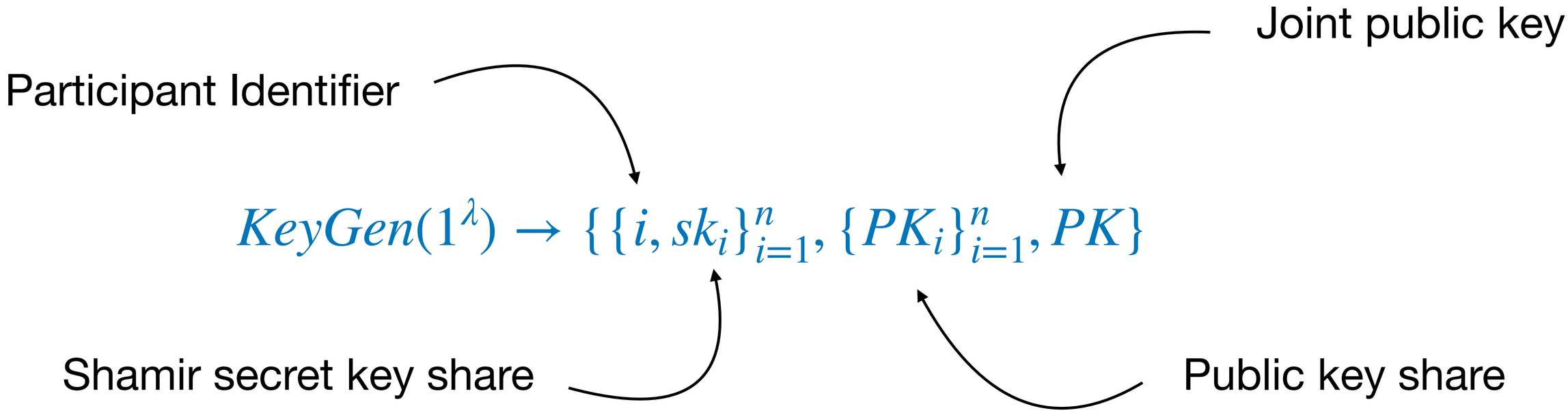
Research Question: Can we design a round-efficient threshold Schnorr scheme?

FROST



Alternative Solution: “Tweak” the Nonce.

Key Generation - Centralized or Decentralized



FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1





PK_2

PK_t



Combine/Verify

$$z = \Sigma_{i=1}^{t} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1

$r_1, s_1 \leftarrow \mathbb{Z}_q$





PK_2



PK_t

Combine/Verify

$$z = \Sigma_{i=1}^{t} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$r_1, s_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}; S_1 = g^{s_1}$



PK_2



PK_t

Combine/Verify

$z = \Sigma_{i=1}^{t} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$r_1, s_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}; S_1 = g^{s_1}$

(R_1, S_1)



PK_2



PK_t

Combine/Verify

$z = \Sigma_{i=1}^{t} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$r_1, s_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}; S_1 = g^{s_1}$

(R_1, S_1)



PK_2



PK_t

Combine/Verify

$z = \Sigma_{i=1}^{t} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$r_1, s_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}; S_1 = g^{s_1}$

(R_1, S_1)

$\{(R_1, S_1) \ldots, (R_t, S_t)\}$



PK_2



PK_t

Combine/Verify

$z = \Sigma_{i=1}^{t} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$r_1, s_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}; S_1 = g^{s_1}$

(R_1, S_1)

$\{(R_1, S_1) \ldots, (R_t, S_t)\}$



PK_2



PK_t

Combine/Verify

$z = \Sigma_{i=1}^{t} z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$$r_1, s_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}; S_1 = g^{s_1}$$

$$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$$

(R_1, S_1)

$\{(R_1, S_1)\ldots, (R_t, S_t)\}$



PK_2



PK_t

Combine/Verify

$$z = \Sigma_{i=1}^{t}\ z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$$r_1, s_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}; S_1 = g^{s_1}$$

$$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$$

$$R = \prod_{i=1}^{t} R_i S_i^{a_i}$$

(R_1, S_1)

$\{(R_1, S_1)\ldots, (R_t, S_t)\}$



PK_2



PK_t

Combine/Verify

$$z = \Sigma_{i=1}^{t}\ z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$$r_1, s_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}; S_1 = g^{s_1}$$

$$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$$

$$R = \prod_{i=1}^{t} R_i S_i^{a_i}$$

$$c = H(PK, R, m)$$

(R_1, S_1) →

← $\{(R_1, S_1)\ldots, (R_t, S_t)\}$



PK_2



PK_t

Combine/Verify

$$z = \Sigma_{i=1}^{t}\ z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$$r_1, s_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}; S_1 = g^{s_1}$$

$$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$$

$$R = \prod_{i=1}^{t} R_i S_i^{a_i}$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + s_1 a + c s k_1 \lambda_1$$

(R_1, S_1) →

← $\{(R_1, S_1) \ldots, (R_t, S_t)\}$



PK_2



PK_t

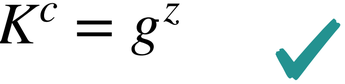
Combine/Verify

$$z = \Sigma_{i=1}^{t} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$



FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$$r_1, s_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}; S_1 = g^{s_1}$$

$$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$$

$$R = \prod_{i=1}^{t} R_i S_i^{a_i}$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + s_1 a + c s k_1 \lambda_1$$

(R_1, S_1)

$\{(R_1, S_1) \ldots, (R_t, S_t)\}$



PK_2



PK_t

Combine/Verify

$$z = \Sigma_{i=1}^{t} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$r_1, s_1 \leftarrow \mathbb{Z}_q$

$R_1 = g^{r_1}; S_1 = g^{s_1}$

(R_1, S_1)



PK_2

$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$

$\{(R_1, S_1)\ldots, (R_t, S_t)\}$



PK_t

$R = \prod_{i=1}^{t} R_i S_i^{a_i}$

$c = H(PK, R, m)$

$z_1 = r_1 + s_1 a + c s k_1 \lambda_1$

$(z_1, \ldots, z_t)$

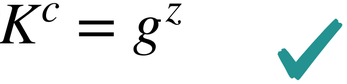
Combine/Verify

$z = \Sigma_{i=1}^{t}\ z_i$

$sig = (R, z)$

$c \leftarrow H(PK, m, R)$

$R \cdot PK^c = g^z$



FROST

[**K**G20, BC**K**MTZ22]

Two-Round threshold Schnorr signature

PK_1



$$r_1, s_1 \leftarrow \mathbb{Z}_q$$

$$R_1 = g^{r_1}; S_1 = g^{s_1}$$

$$a_i = H(PK, i, m, \{(i, R_i, S_i)\}_{i=1}^{t})$$

$$R = \prod_{i=1}^{t} R_i S_i^{a_i}$$

$$c = H(PK, R, m)$$

$$z_1 = r_1 + s_1 a + c s k_1 \lambda_1$$

(R_1, S_1) →

← $\{(R_1, S_1) \ldots, (R_t, S_t)\}$

↓

$(z_1, \ldots, z_t)$



PK_2



PK_t

Combine/Verify

$$z = \Sigma_{i=1}^{t} z_i$$

$$sig = (R, z)$$

$$c \leftarrow H(PK, m, R)$$

$$R \cdot PK^c = g^z$$

✓

Reminder: $sk = \Sigma_{i=1}^{t} \lambda_i sk_i$

FROST in Practice







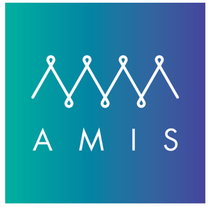














serai-dex/**serai**







CFRG D. Connolly
Internet-Draft Zcash Foundation
Intended status: Informational C. Komlo
Expires: 28 July 2023 University of Waterloo, Zcash Foundation
I. Goldberg
University of Waterloo
C. A. Wood
Cloudflare
24 January 2023

Two-Round Threshold Schnorr Signatures with FROST
draft-irtf-cfrg-frost-12





jesseposner/
FROST-BIP340



BIP340 compatible implementation of Flexible Round-Optimized Schnorr Threshold Signatures (FROST). This work is made possible with the support of Brink.

2 Contributors · 2 Issues · 20 Stars · 3 Forks

FROST in Practice





Why is FROST Popular in Practice?

Why is FROST Popular in Practice?

- **Simple!**

Why is FROST Popular in Practice?

- **Simple!**
- One-round with preprocessing

Why is FROST Popular in Practice?

- **Simple!**
- One-round with preprocessing
- Preprocessing round is independent of message and signing set

Why is FROST Popular in Practice?

- **Simple!**
- One-round with preprocessing
- Preprocessing round is independent of message and signing set
- Straightforward identifiable abort (no additional ZK proof required

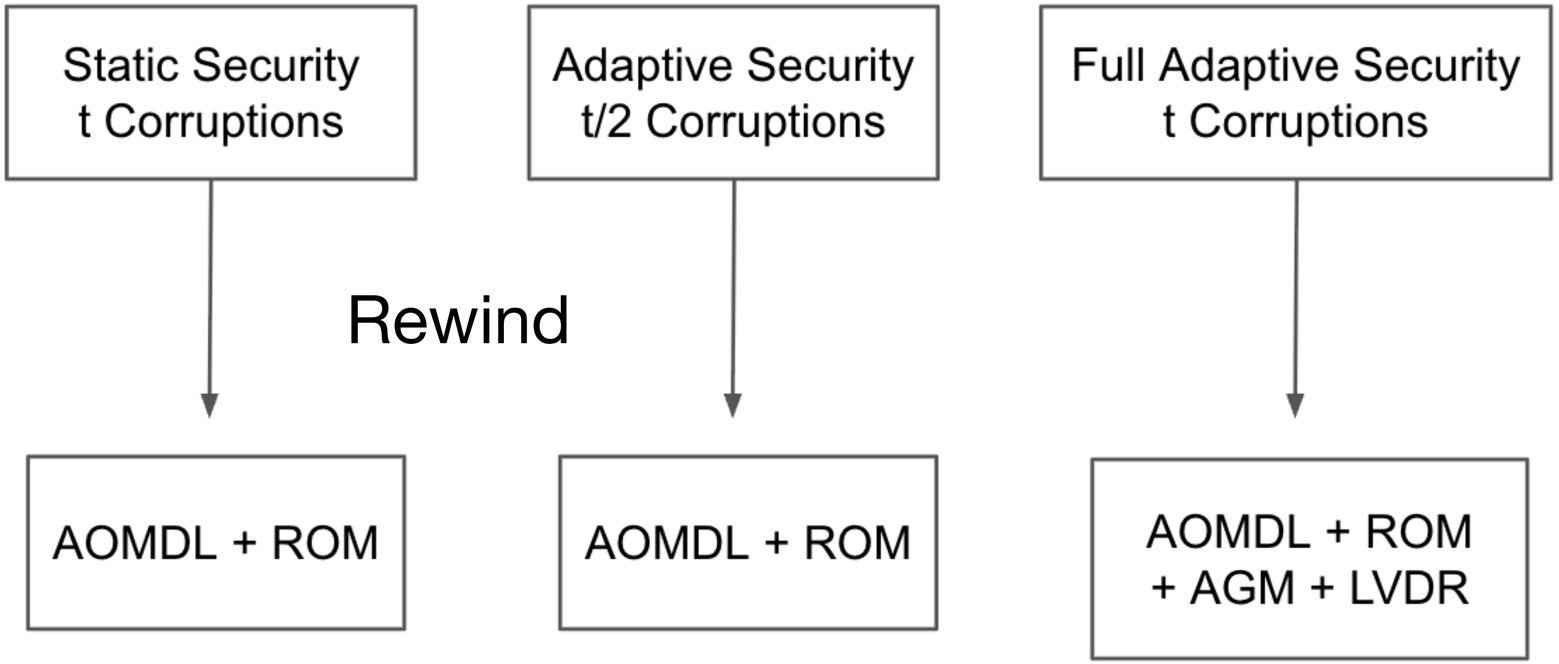
Why is FROST Popular in Practice?

- **Simple!**
- One-round with preprocessing
- Preprocessing round is independent of message and signing set
- Straightforward identifiable abort (no additional ZK proof required

$$R_i \cdot PK_i^c \stackrel{?}{=} g^{z_i}$$

Security of FROST

[CK**K**TZ25]

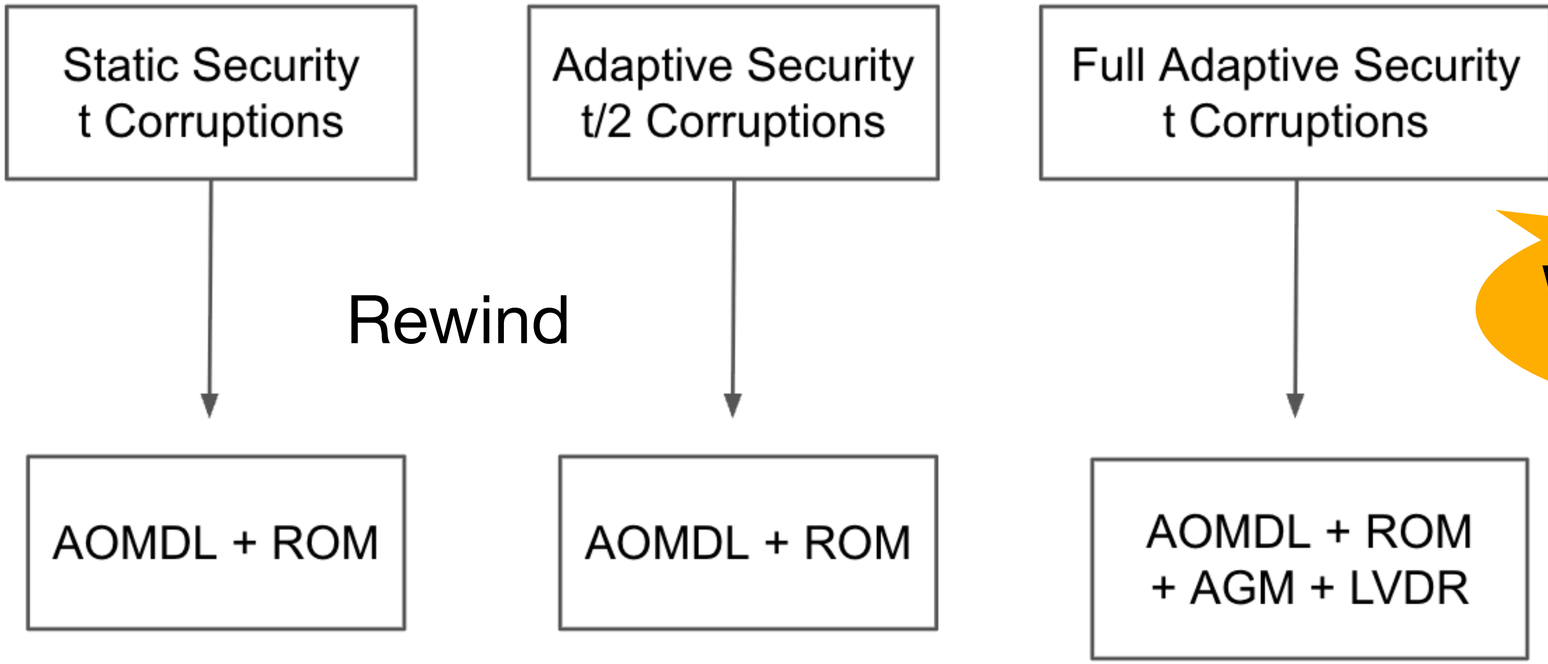


AOMDL = Algebraic One-More Discrete Logarithm
AGM = Algebraic Group Model
LDVR: Low-Dimensional Vector Representation Assumption

(Threshold = $t + 1$)

Security of FROST

[CK**K**TZ25]



AOMDL = Algebraic One-More Discrete Logarithm
AGM = Algebraic Group Model
LDVR: Low-Dimensional Vector Representation Assumption

(Threshold = $t + 1$)

Submission

Choices

Choices

- Key generation is agnostic (centralized or decentralized)

Choices

- Key generation is agnostic (centralized or decentralized)
- Two-round protocol

Choices

- Key generation is agnostic (centralized or decentralized)
- Two-round protocol
- One-round with preprocessing is possible (with additional implementation choices)

Takeaways

Takeaways

- FROST is a practical two-round threshold Schnorr signature scheme with significant real-world use.

Takeaways

- FROST is a practical two-round threshold Schnorr signature scheme with significant real-world use.
- Compatible with any key generation that outputs Shamir secret-shared keys.

Takeaways

- FROST is a practical two-round threshold Schnorr signature scheme with significant real-world use.
- Compatible with any key generation that outputs Shamir secret-shared keys.
- Provably secure in both static and (non-trivial) adaptive security models.

Takeaways

- FROST is a practical two-round threshold Schnorr signature scheme with significant real-world use.
- Compatible with any key generation that outputs Shamir secret-shared keys.
- Provably secure in both static and (non-trivial) adaptive security models.

Thank you!